M.Stat Project Report(Final Report)

Your Name: Sounak Das
Roll Number: MB2331
Guide Name: Prof. Smarajit Bose
Institution Name: Indian Statistical Institute, Kolkata

May 16, 2025

Abstract

Class imbalance poses significant challenges in classification tasks, often leading to poor predictive performance for minority classes. This project initially focused on classical oversampling strategies, particularly SMOTE and its improved variants such as k-means SMOTE and Borderline SMOTE. These methods aim to balance class distributions by generating synthetic data in safe or informative regions of the feature space. While effective for tabular data, their limitations become apparent when extended to graph-structured datasets, where node relationships and topology are crucial for learning.

To address this, the study extends the SMOTE framework to graphs by exploring GraphSMOTE and its advanced variants—GTS, FG-SMOTE, and ESA-GCN. These methods introduce structure-aware interpolation, edge generation, fairness-aware sampling, and robustness to noise through attention mechanisms. Comprehensive experiments on real-world graph datasets, including Cora, PubMed, YelpChi, and Credit, demonstrate improvements in both classification accuracy and fairness. The project concludes that graph-specific oversampling strategies, when thoughtfully designed, can significantly enhance performance and equity in node classification tasks.

Introduction

The class imbalance problem arises in machine learning when classification tasks involve unequally represented classes, common in fraud detection, rare disease diagnosis, and satellite image analysis. Standard classifiers, often biased toward the majority class, struggle with minority class predictions, especially when misclassification costs vary.

To address imbalance, data-level methods modify training data via undersampling or oversampling. While undersampling risks information loss, simple oversampling can cause overfitting. SMOTE mitigates this by generating synthetic samples through interpolation. Informed oversampling strategies further improve classification by focusing on sparse regions, safe areas, or decision boundaries. This work explores SMOTE and its variants to enhance classification in imbalanced datasets.

SMOTE Description

SMOTE is an over-sampling approach in which the minority class is over-sampled by creating "synthetic" examples rather than by over-sampling with replacement. This approach is inspired by a technique that proved successful in handwritten character recognition (Ha & Bunke, 1997).
We generate synthetic examples in a less application-specific manner, by operating in "feature space" rather than "data space". The minority class is over-sampled by taking each minority class sample

and introducing synthetic examples along the line segments joining any/all of the k minority class nearest neighbors. Depending upon the amount of over-sampling required, neighbors from the k nearest neighbors are randomly chosen. Our implementation currently uses five nearest neighbors. For instance, if the amount of over-sampling needed is 200 %, only two neighbors from the five nearest neighbors are chosen and one sample is generated in the direction of each. Synthetic samples are generated in the following way: Take the difference between the feature vector (sample) under consideration and its nearest neighbor. Multiply this difference by a random number between 0 and 1, and add it to the feature vector under consideration. This causes the selection of a random point along the line segment between two specific features. This approach effectively forces the decision region of the minority class to become more general.

The key idea of SMOTE is to introduce synthetic examples. This new data is created by interpolation between several minority class instances that are within a defined neighborhood. For this reason, the procedure is said to be focused on the "feature space" rather than on the "data space", in other words, the algorithm is based on the values of the features and their relationship, instead of considering the data points as a whole. This has also led to studying the theoretical relationship between original and synthetic instances must be analyzed in depth, including the data dimensionality. Some properties such as variance and correlation in the data and feature space, as well as the relationship between training and test examples distribution must be considered. Here are some images for that.

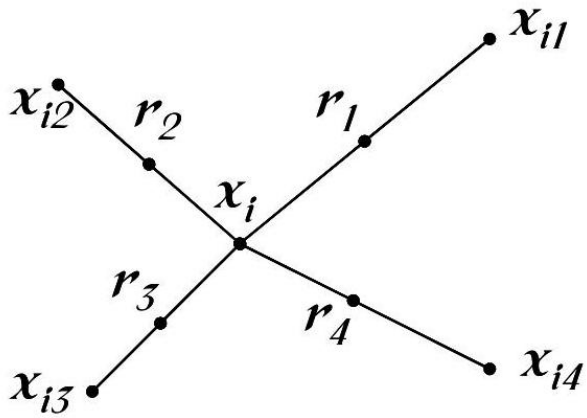

Figure 1: An illustration of how to create the synthetic data points in the SMOTE algorithm

An x_i minority class instance is selected as basis to create new synthetic data points. Based on a distance metric, several nearest neighbors of the same class (points x_{i1} to x_{i4}) are chosen from the training set. Finally, a randomized interpolation is carried out in order to obtain new instances r_1 to r_4.

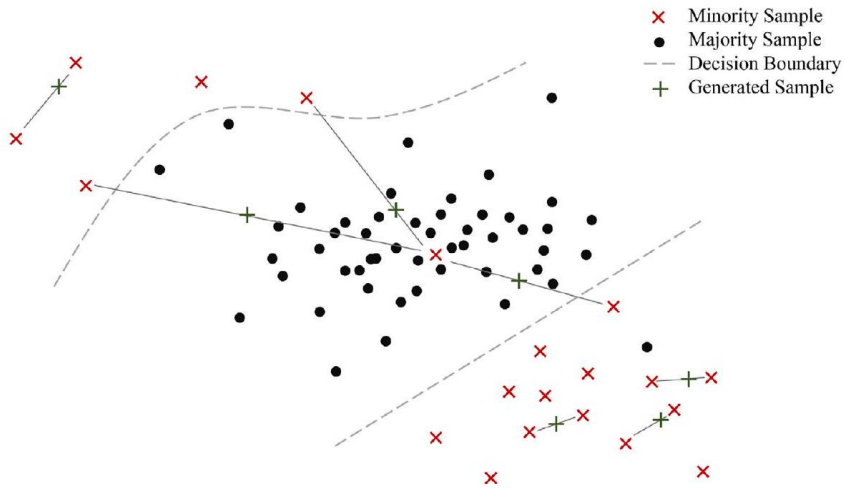

Figure 2: SMOTE may generate minority samples in majority regions in the presence of noise. Most non-noise samples are generated in already dense minority areas, contributing to within-class imbalance.

Algorithm 1 SMOTE Algorithm

```
function SMOTE(T, N, k)
    Input:
    T: Number of minority class examples
    N: Amount of oversampling (percentage)
    k: Number of nearest neighbors
    Output: (N/100) × T synthetic minority class samples
    Variables:
    Sample[][]: Array for original minority class samples
    Synthetic[]: Array for synthetic samples
    newindex: Counter for synthetic samples (initialized to 0)
    if N < 100 then
        Randomly shuffle the T minority class samples
        T = (N/100) × T
        N = 100
    end if
    N = ⌊N/100⌋                    ▷ Ensure SMOTE is applied in multiples of 100
    for i = 1 to T do
        Compute k nearest neighbors for sample i
        Store the indices of k nearest neighbors in nnarray
        Call POPULATE(N, i, nnarray)
    end for
end function
```

Drawbacks of SMOTE

However, SMOTE has certain limitations when dealing with class imbalance and noise, as illustrated in 2. One major drawback arises from the uniform random selection of minority instances for oversampling. While this approach effectively addresses between-class imbalance, it fails to consider within-class imbalance and small disjuncts. As a result, densely populated minority regions tend to get oversampled even further, whereas sparsely populated minority areas remain underrepresented.

Another significant concern is that SMOTE does not differentiate between overlapping class regions and safe areas. This makes the method prone to generating synthetic samples in noisy regions, which can mislead classifiers. Additionally, if a noisy minority sample—positioned among majority class in-

Algorithm 2 Function to Generate Synthetic Samples

```
function POPULATE(N, i, nnarray)
    Input: N, i, nnarray                ▷ #Instances to create, original sample index, nearest neighbors
    Output: N new synthetic samples in Synthetic array
    while N ≠ 0 do
        nn = random(1, k)
        for attr = 1 to numattrs do                    ▷ numattrs = Number of attributes
            dif = Sample[nnarray[nn]][attr] − Sample[i][attr]
            gap = random(0, 1)
            Synthetic[newindex][attr] = Sample[i][attr] + gap · dif
        end for
        newindex ← newindex + 1
        N ← N − 1
    end while
end function
```

stances—is selected for interpolation, SMOTE may amplify the noise instead of improving class balance.

Borderline-1 SMOTE & Borderline-2 SMOTE

Borderline-1 SMOTE and Borderline-2 SMOTE are variations of SMOTE designed to focus on minority class samples near the decision boundary. In Borderline-1 SMOTE, only minority samples misclassified by a k-nearest neighbors (k-NN) classifier are oversampled, as they are more likely to be crucial for decision boundaries. Borderline-2 SMOTE extends this by considering both minority and majority class neighbors; if most neighbors belong to the majority class, synthetic samples are generated closer to the minority class to reinforce class separation. Both methods aim to enhance classification performance by improving decision boundaries while reducing noise.One can view this paper which for further references B-1 SMOTE & B-2 SMOTE

k-Means SMOTE

This method combines k-means clustering with SMOTE to rebalance skewed datasets while avoiding noise by oversampling only in safe areas. It addresses both between-class and within-class imbalance by prioritizing sparse minority regions. Unlike other methods, it distributes synthetic samples based on cluster density, ensuring more samples in sparse areas. Clustering is done irrespective of class labels, enhancing safe oversampling detection. Additionally, overfitting is minimized by generating new samples rather than duplicating them.

K-means SMOTE consists of three steps: clustering, filtering, and oversampling. In the clustering step, the input space is clustered into k groups using k-means clustering. The filtering step selects clusters for oversampling, retaining those with a high proportion of minority class samples. It then distributes the number of synthetic samples to generate, assigning more samples to clusters where minority samples are sparsely distributed. Finally, in the oversampling step, SMOTE is applied in each selected cluster to achieve the target ratio of minority and majority instances. The algorithm is illustrated in 3

The k-means algorithm is a popular iterative method of finding naturally occurring groups in data which can be represented in a Euclidean space. It works by iteratively repeating two instructions: First, assign each observation to the nearest of k cluster centroids. Second, update the position of the centroids so that they are centered between the observations assigned to them. The algorithm converges when no more observations are reassigned. It is guaranteed to converge to a typically local optimum in a finite

Algorithm 3 Proposed method based on k-means and SMOTE

Input:
X: Matrix of observations
y: Target vector
n: Number of samples to be generated
k: Number of clusters for k-means
irt: Imbalance ratio threshold
knn: Number of nearest neighbors for SMOTE
de: Exponent used for density computation (default: number of features in X)
Output: Set of synthetic samples
▷ **Step 1: Cluster the input space and filter clusters dominated by the minority class**
$clusters \leftarrow kmeans(X, k)$
$filtered_clusters \leftarrow \varnothing$
for each $c \in clusters$ **do**
  $imbalance_ratio \leftarrow \frac{majority_count(c)+1}{minority_count(c)+1}$
  if $imbalance_ratio < irt$ **then**
    $filtered_clusters \leftarrow filtered_clusters \cup \{c\}$
  end if
end for
▷ **Step 2: Compute sampling weights based on minority density**
for each $f \in filtered_clusters$ **do**
  $avg_minority_dist(f) \leftarrow mean(euclidean\ distances(f))$
  $density_factor(f) \leftarrow \frac{minority_count(f)}{avg_minority_dist(f) \times de}$
  $sparsity_factor(f) \leftarrow \frac{1}{density_factor(f)}$
end for
$sparsity_sum \leftarrow \sum_{f \in filtered_clusters} sparsity_factor(f)$
for each $f \in filtered_clusters$ **do**
  $sampling_weight(f) \leftarrow \frac{sparsity_factor(f)}{sparsity_sum}$
end for
▷ **Step 3: Oversample each filtered cluster using SMOTE**
$generated_samples \leftarrow \varnothing$
for each $f \in filtered_clusters$ **do**
  $num_samples \leftarrow round(n \times sampling_weight(f))$
  $generated_samples \leftarrow generated_samples \cup SMOTE(f, num_samples, knn)$
end for
return $generated_samples$

number of iterations . For large datasets where k-means may be slow to converge, more efficient implementations could be used for the clustering step of the k-means SMOTE, such as mini-batch k-means.

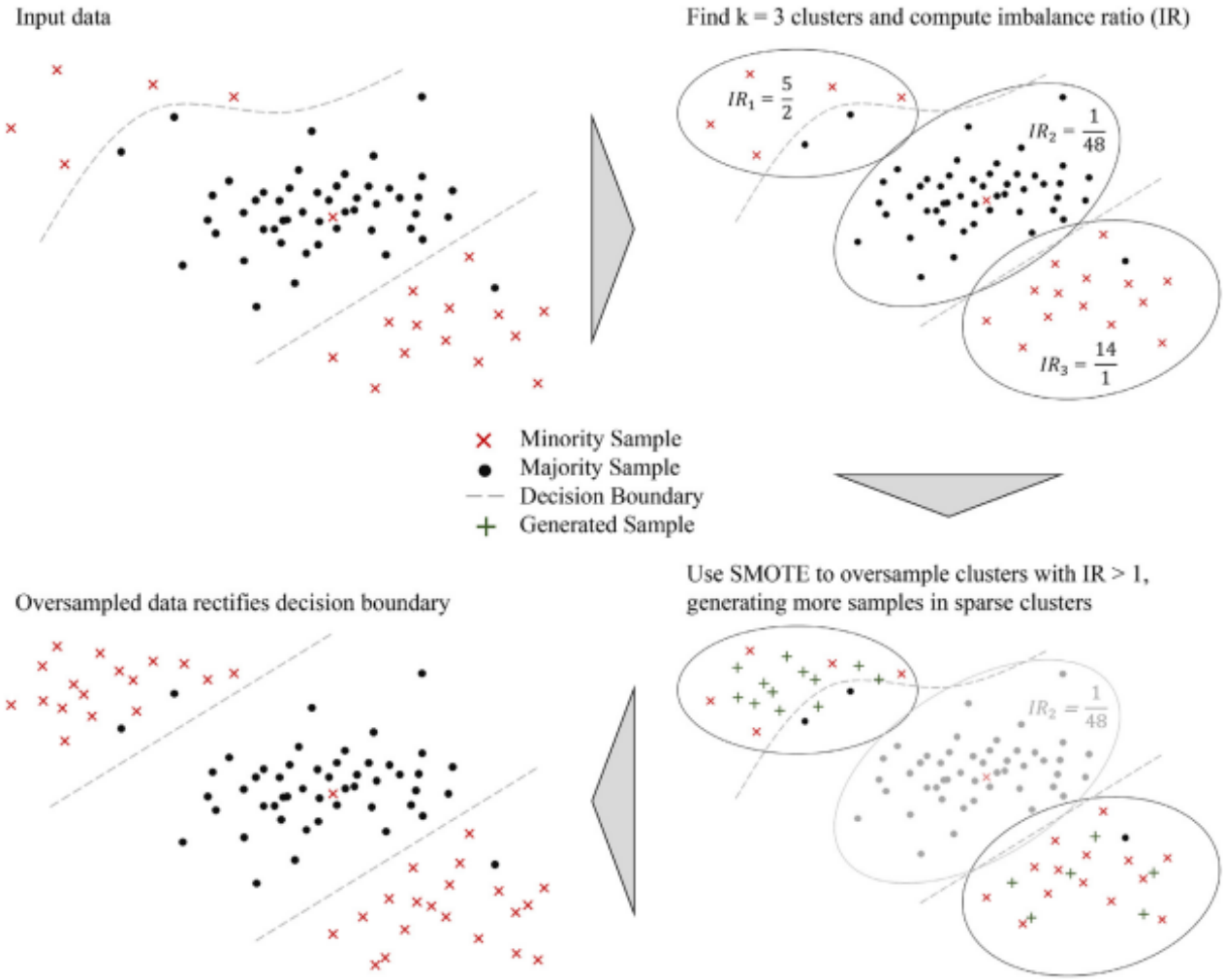

Figure 3: K-means SMOTE oversamples safe areas and combats within-class imbalance

The filter step selects clusters for oversampling based on their minority-majority ratio, prioritizing safer, minority-dominant regions. To ensure balanced distribution, sparse minority clusters receive more synthetic samples than dense ones. By default, clusters with at least 50% minority instances are selected, but this can be adjusted via the imbalance ratio threshold (irt) defined as $\frac{\text{majority count}(c)+1}{\text{minority count}(c)+1}$.When the imbalance ratio threshold is increased, cluster choice is more selective and a higher proportion of minority instances is required for a cluster to be selected. On the other hand, lowering the threshold loosens the selection criterion, allowing clusters with a higher majority proportion to be chosen.

To determine the distribution of samples to be generated, filtered clusters are assigned sampling weights between zero and one. A high sampling weight corresponds to a low density of minority samples and yields more generated samples. To achieve this, the sampling weight depends on how dense a single cluster is compared to how dense all selected clusters are on average. Note that when measuring a cluster's density, only the distances among minority instances are considered. The computation of the sampling weight may be expressed by means of five sub-computations:

1. For each filtered cluster f, calculate the Euclidean distance matrix, ignoring majority samples.

2. Compute the mean distance within each cluster by summing all non-diagonal elements of the distance matrix, then dividing by the number non-diagonal elements.

3. To obtain a measure of density, divide each cluster's number of minority instances by its average minority distance raised to the power of the number of features m : density $(f) = \frac{\text{minority count } (f)}{\text{average minority distance } (f)^m}$.

4. Invert the density measure as to get a measure of sparsity, i.e. $\text{sparsity}(f) = \frac{1}{\text{density}(f)}$.

5. The sampling weight of each cluster is defined as the cluster's sparsity factor divided by the sum of all clusters' sparsity factors.

The proposed method differs from related techniques by clustering the entire dataset, independent of class labels. This unsupervised approach helps identify overlapping class regions and avoids unsafe oversampling. Since k-means typically forms clusters of similar size but varying density , this strategy effectively addresses within-class imbalance. Additionally, using SMOTE mitigates overfitting, a common issue with random oversampling.

Limit cases

SMOTE and random oversampling can be seen as special cases of k-means SMOTE. In k-means SMOTE, data is clustered using k-means, and selected clusters are oversampled with SMOTE. When $k = 1$, all data points form a single cluster. To classify it as a minority cluster, the imbalance ratio threshold must match the dataset's imbalance. For example, in a dataset with 100 minority and 10,000 majority samples, the threshold must be at least $\frac{10{,}000+1}{100+1} \approx 99.02$. Oversampling this single cluster with SMOTE is equivalent to applying SMOTE to the entire dataset. Setting the threshold to infinity achieves the same effect.

If SMOTE interpolated a single point instead of two, it would replicate the original data, effectively mimicking random oversampling. This can be achieved by setting SMOTE's " k nearest neighbors" parameter to zero, if supported.

This flexibility makes k-means SMOTE highly practical, as it encompasses both SMOTE and random oversampling. Hyperparameter tuning can explore configurations for both methods, ensuring k-means SMOTE performs at least as well as the best of the two. Table 1 summarizes the parameter settings that replicate both algorithms.

	k	irt	knn
SMOTE	1	∞	
Random oversampling	1	∞	0

Table 1: Parameter settings to replicate SMOTE and random oversampling.

Comparison Methodology

Selection of Metric

The choice of metric depends to a great extent on the goal their user seeks to achieve. In certain practical tasks, one specific aspect of classification may be more important than another (e.g. in medical diagnoses, false negatives are much more critical than false positives). However, to determine a general ranking

among oversamplers, no such focus should be placed. Therefore, the following unweighted metrics are chosen for the evaluation.

- g-mean
- F1-score
- AUPRC

The F1-score, or F-measure, is the (often weighted) harmonic mean of precision and recall. In other terms, the indicator rates both the completeness and exactness of positive predictions.

$$F1 = \frac{(1+\alpha) \times (\text{ sensitivity } \times \text{ precision })}{\text{sensitivity } + \alpha \times \text{ precision}} = \frac{(1+\alpha) \times \left(\frac{TP}{P} \times \frac{TP}{PP}\right)}{\frac{TP}{P} + \alpha \times \frac{TP}{PP}} \tag{2}$$

The geometric mean score, also referred to as g-mean or g-measure, is defined as the geometric mean of sensitivity and specificity. The two components can be regarded as per-class accuracy. The g-measure aggregates both metrics into a single value in [0,1], assigning equal importance to both [23,26].

$$g- \text{ mean } = \sqrt{\text{ sensitivity } \times \text{ specificity }} = \sqrt{\frac{TP}{P} \times \frac{TN}{N}} \tag{3}$$

Precision-recall (PR) diagrams plot the precision of a classifier as a function of its minority accuracy. Classifiers outputting class membership confidences (i.e. continuous values in $[0,1]$) can be plotted as multiple points in discrete intervals, resulting in a PR curve. Commonly, the area under the precision-recall curve (AUPRC) is computed as a single numeric performance metric .

Oversamplers

The following list enumerates the oversamplers used as a benchmark for the evaluation of the proposed method, along with the set of hyperparameters used for each. The optimal imbalance ratio is not obvious and has been discussed by other researchers [5, 13, 40]. This work aims at creating comparability among oversamplers; consequently, it is most important that oversamplers achieve the same imbalance ratio. Therefore, all oversampling methods were parametrized to generate as many instances as necessary so that minority and majority classes count the same number of samples.

- random oversampling
- SMOTE - knn $\in \{3, 5, 20\}$
- borderline-SMOTE1 - knn $\in \{3, 5, 20\}$
- borderline-SMOTE2 - knn $\in \{3, 5, 20\}$
- k-means SMOTE $-k \in \{2, 20, 50, 100, 250, 500\}$ - $knn \in \{3, 5, 20, \infty\}$ - irt $\in \{1, \infty\}$ - de $\in \{0, 2,$ number of features $\}$
- no oversampling

Classifiers

The following list enumerates the classifiers used in this study along with a set of values for their respective hyperparameters.

- **LR:** Logistic regression (LR) is a generalized linear model which can be used for binary classification. Fitting the model is an optimization problem which can be solved using simple optimizers which require no hyperparameters to be set. Consequently, results achieved by LR are easily reproducible, while also constituting a baseline for more sophisticated approaches.

- **KNN** $-K\{3, 5, 8\}$ k-nearest neighbor assigns an observation to the class most of its nearest neighbors belong to. How many neighbors are considered is determined by the method's hyperparameter k.

- **GBM** - number of trees $\in \{50, 100, 200\}$ Gradient boosting over decision trees, or simply (GBM), is an ensemble technique used for classification. In the case of binary classification, one shallow decision tree is induced at each stage of the algorithm. Each tree is fitted to observations which could not be correctly classified by decision trees of previous stages. Predictions of GBM are made by majority vote of all trees. In this way, the algorithm combines several simple models (referred to as weak learners) to create one effective classifier. The number of decision trees to generate, which in binary classification is equal to the number of stages, is a hyperparameter of the algorithm

Data Sets

K-means SMOTE is evaluated using 12 imbalanced datasets from the UCI Machine Learning Repository. Multi-class datasets were binarized via a one-versus-rest approach, with the smallest class as the minority. To further increase imbalance, each dataset was randomly undersampled, generating up to six additional versions with imbalance ratios approximately multiplied by $2, 4, 6, 10, 15$, and 20, ensuring at least eight minority samples remained. Additionally, 19 datasets from the KEEL repository [1] and ten artificial variations of the "MADELON" dataset from scikit-learn [38] were included.

Table 2 summarizes the datasets used for evaluation, excluding undersampled versions. In Section 5, figures append the undersampling factor to dataset names.

To illustrate k-means SMOTE's properties, three two-dimensional binary datasets (A, B, and C) were created, consisting of observations in three partially noisy clusters. All the data set can be found here : All Data Sets

Experimental framework

The proposed method is evaluated using specified oversamplers, metrics, datasets, and classifiers. Results are averaged over five runs of 5-fold cross-validation, with standard deviations also reported.

A grid search optimizes hyperparameters by testing all possible combinations for each classifier and oversampler. Each configuration is scored across all metrics, and the best values are recorded. For clarity, an example with one oversampler and classifier is provided.

- SMOTE
- $knn \in \{3, 6\}$
- GBM
- number of trees $\in \{10, 50\}$

The oversampling method SMOTE is run two times, generating two different sets of training data. For each set of training data, the classifier LR is executed twice. Therefore, the classifier will be executed four times. The possible combinations are:

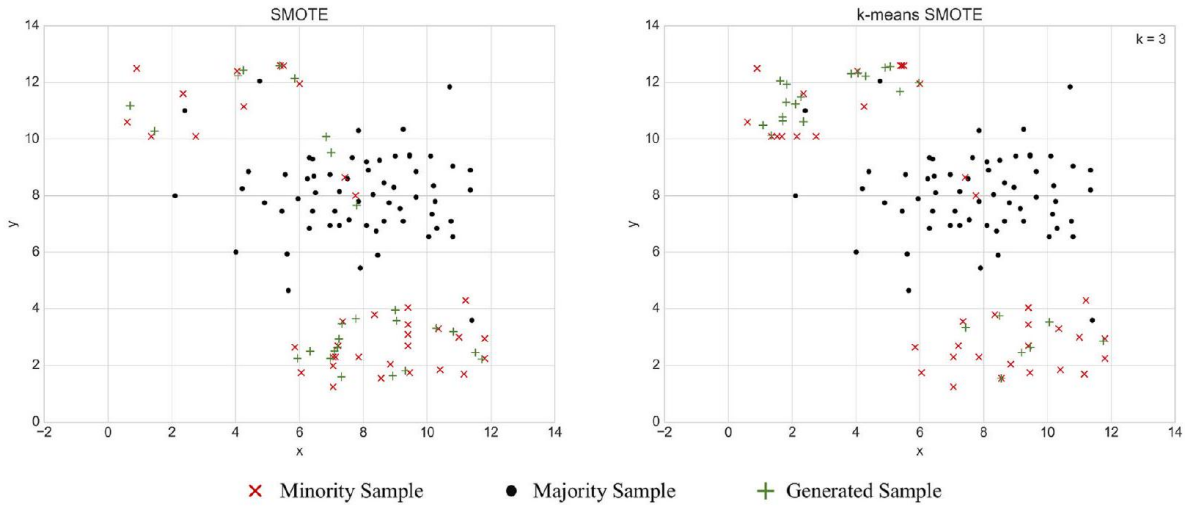

Figure 4: Samples generated through oversampling toy dataset A with SMOTE (left) and k-means SMOTE (right)

- $knn = 3$, number of trees = 10
- knn = 3, number of trees = 50
- $knn = 6$, number of trees = 10
- $knn = 6$, number of trees = 50

If two metrics are used for scoring, both metrics score all four runs; the best result is saved. Note that one metric may find that the combination $knn = 3$, number of trees = 10 is best, while a different combination might be best considering another metric. In this way, each oversampler and classifier combination is given the chance to optimize for each metric.

Results

Before analyzing real datasets, we first tested the method on toy datasets. Figure 4, 5 and 6 show the resulting outputs. Using two-dimensional toy datasets, we illustrate how k-means SMOTE improves upon plain SMOTE. Figure 4, 5 and 6 depict the results on simple binary datasets with two to three slightly noisy clusters. Overall, these experiments show that the proposed method effectively balances classes while minimizing noise generation.

Instead of ranking classification algorithms, we ranked the oversamplers using the **Friedman test and Holm's method,** which are commonly used in imbalanced classification research due to their non-parametric nature.

Ranks were assigned based on cross-validated scores, with rank one for the best-performing and rank six for the worst. Each method's rank was averaged across datasets and experiment repetitions, yielding a mean rank in the range [1.0,6.0]. The Friedman test confirmed that rank differences among oversamplers were statistically significant ($\alpha = 0.05$)

Consequently, Holm's step-down procedure was applied with the proposed method as the control. This post-hoc test adjusted significance levels to account for multiple comparisons. The null hypothesis in each pair-wise comparison of the control method(we used k-Means SMOTE as the control method)

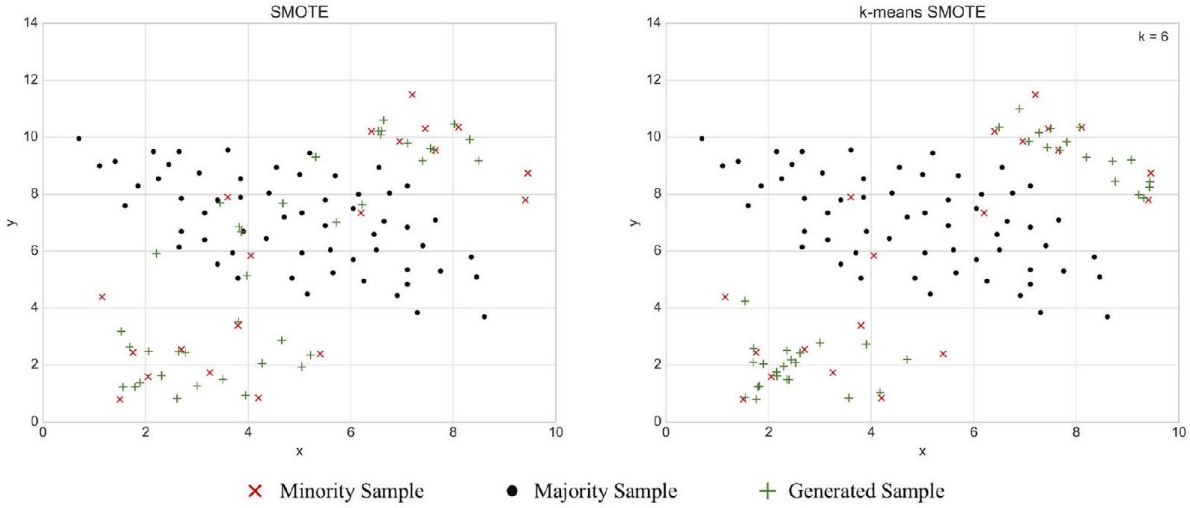

Figure 5: Samples generated through oversampling toy dataset B with SMOTE (left) and k-means SMOTE (right)

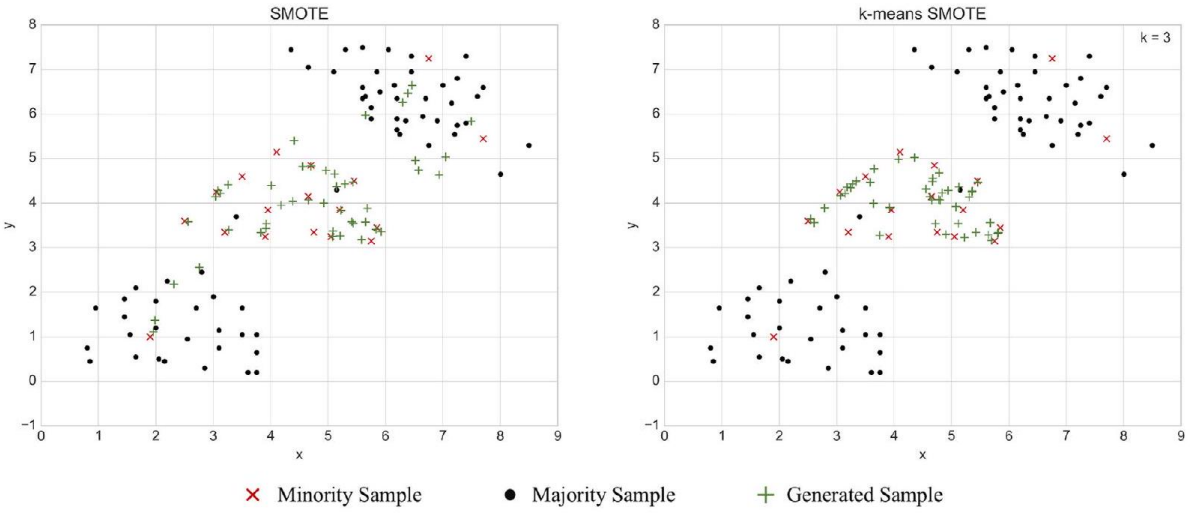

Figure 6: Samples generated through oversampling toy dataset C with SMOTE (left) and k-means SMOTE (right)

Table 2: Results of Holm's test with k-means SMOTE as the control method.

Classifier $\alpha_{0.05}$	GBM		KNN		LR	
	Oversampler	p-value	Oversampler	p-value	Oversampler	p-value
Metric: AUPRC						
0.01	None	$1.47E-30$	B2-SMOTE	4.58E-32	B2-SMOTE	4.36E-21
$0.01\overline{25}$	Random 0.	$9.18E-21$	Random 0.	$3.47E-25$	B1-SMOTE	$9.58E-18$
$0.01\overline{66}$	B2-SMOTE	$8.56E-14$	B1-SMOTE	8.90E-24	Random 0.	$3.17E-16$
$0.02\overline{5}$	B1-SMOTE	$2.07E-13$	None	$1.40E-16$	None	$4.07E-11$
0.05	SMOTE	$3.61E-10$	SMOTE	$1.40E-16$	SMOTE	$6.03E-11$
Metric: F1						
0.01	None	$5.73E-41$	None	1.03E-33	None	2.12E-33
$0.01\overline{25}$	Random 0.	7.29E-24	B2-SMOTE	1.00E-25	B2-SMOTE	7.88E-25
$0.01\overline{66}$	B2-SMOTE	$4.71E-14$	Random 0.	1.55E-13	Random 0.	7.94E-22
0.025	B1-SMOTE	$2.40E-11$	SMOTE	$4.29E-13$	B1-SMOTE	$4.72E-12$
0.05	SMOTE	$9.71E-10$	B1-SMOTE	$2.10E-11$	SMOTE	1.84E-11
Metric: g-mean						
0.01	None	4.98E-44	None	$4.73E-56$	None	1.39E-48
$0.01\overline{25}$	Random 0.	4.88E-24	Random 0.	6.92E-20	B1-SMOTE	$5.29E-11$
$0.01\overline{66}$	B1-SMOTE	$3.58E-12$	B2-SMOTE	$8.07E-18$	B2-SMOTE	$1.10E-09$
0.025	SMOTE	$4.03E-05$	B1-SMOTE	1.63E-14	Random 0.	$1.30E-08$
0.05	B2-SMOTE	$6.65E-04$	SMOTE	$5.46E-06$	SMOTE	$1.21E-06$

and another oversampler is that the control method does not perform better than the other method. 2 presents the results, showing that the null hypothesis was rejected for all oversamplers ($\alpha = 0.05$), indicating the proposed method outperforms all others.

Generally, it can be observed that - aside from the proposed method - SMOTE, borderline-SMOTE1, and borderlineSMOTE2 typically achieve the best results, while no oversampling frequently earns the worst rank. Remarkably, LR and KNN achieve a similar rank without oversampling as with SMOTE with regard to AUPRC, while both are only dominated by k-means SMOTE. This indicates that the proposed method may improve classifier performance even in situations where SMOTE is not able to achieve any improvement versus the original training data

Conclusion

Imbalanced data challenges classifiers, as duplicating minority instances leads to overfitting while synthetic generation risks noise. Most oversamplers also ignore within-class imbalances. Effective oversampling should focus on safe regions and balance minority distributions.

The proposed k-means SMOTE clusters data to identify safe minority-dense areas, preventing noise generation. Sparse minority clusters receive more synthetic samples, reducing within-class imbalance. By generating new data instead of duplicating, it also mitigates overfitting.

Experiments show k-means SMOTE consistently outperforms SMOTE, borderline-SMOTE, and random oversampling, especially in moderately complex tasks. It remains efficient, leveraging widely available k-means and SMOTE libraries, with a Python implementation on Python File

Applicable across domains like fraud detection and disease diagnosis, future work may refine hyperparameter selection, particularly optimizing k based on dataset properties.

Endterm Report

Extension Towards Graph-based Oversampling Techniques

From Classical SMOTE to GraphSMOTE

The Synthetic Minority Oversampling Technique (SMOTE) has been effective in addressing class imbalance in i.i.d. (independent and identically distributed) tabular data by interpolating new samples between existing minority instances. However, when applied to graph-structured data—such as social networks, citation graphs, and transaction networks—SMOTE fails to capture the complex relational dependencies and topological patterns inherent to graphs.

- It cannot preserve graph structure, leading to disconnected synthetic nodes.
- It does not account for message passing mechanisms in GNNs, which depend on both node features and graph topology.

To overcome these limitations, **GraphSMOTE** was proposed. It extends SMOTE by operating in the learned embedding space of a Graph Neural Network (GNN), and adds an *edge generator* to reconnect synthetic nodes meaningfully within the graph.

Why We Need More Than Just GraphSMOTE

Although GraphSMOTE represents a significant advance, several real-world constraints call for further refinement. These challenges lead to three prominent extensions of GraphSMOTE, each addressing a different shortcoming:

1. Graph Tran-SMOTE (GTS): Structure-Aware Oversampling

Motivation: GraphSMOTE may suffer from *over-smoothing*, especially in deep GNNs, where minority class features are diluted through excessive neighbor aggregation.

What It Adds:

- A *subgraph extractor* that focuses on localized graph neighborhoods (e.g., "fraud rings" or "small gangs").
- A *structure-aware transformer* that integrates both node attributes and structural information through self-attention.
- A specialized edge generator trained to reconnect synthetic nodes based on their subgraph context.

Resulting Benefit: More expressive node representations and improved classification for structurally marginalized classes in real datasets like *YelpChi* and *Amazon*.

2. FG-SMOTE: Fairness-Aware Oversampling

Motivation: Standard oversampling techniques may exacerbate bias by over-representing certain sensitive subgroups (e.g., gender, race) or by creating biased structural links.

What It Adds:

- A *Sensitive Information De-identification Module* to desensitize embeddings.
- A *Global Sampling Strategy* that selects interpolation anchors across subgroups to increase diversity.

- A *Fair Link Predictor* that avoids reinforcing structural segregation.

Resulting Benefit: Balances subgroup representation without sacrificing classification performance, ideal for high-stakes applications like credit scoring or healthcare.

3. ESA-GCN: Noise-Robust Oversampling with Attention

Motivation: In noisy real-world graphs, directly interpolating between misclassified or mislabeled nodes can degrade model performance.

What It Adds:

- *Edited Nearest Neighbor (ENN)* pruning to remove noisy nodes before oversampling.
- *SMOTE-based generation* in the GNN embedding space.
- An *attention-based edge generator* that adaptively learns which neighbors are trustworthy.

Resulting Benefit: Enhances stability and performance in noisy or partially mislabeled graph datasets, such as fraud transaction logs or user behavior graphs.

Summary: Three Complementary Extensions

Model	Key Feature	What It Solves
GraphSMOTE	Interpolation in embedding space + Edge generator	Structure-aware oversampling for imbalanced node classification
GTS	Subgraph extractor + Transformer + Local edge gen	Over-smoothing and weak local structure learning
FG-SMOTE	Sensitive info removal + Fair link prediction	Bias and subgroup discrimination in node generation
ESA-GCN	ENN pruning + Attention-based edge generation	Robustness against noisy samples and mislabeled neighbors

Table: Overview of GraphSMOTE variants and their unique contributions

This structured evolution highlights the growing need for task-specific, fairness-aware, and noise-resilient graph oversampling techniques. In the following sections, we explore these frameworks mathematically and implement them in real datasets.

GraphSMOTE: Methodology and Mathematical Framework

Problem Formulation

We are given a graph $G = (V, E, X, Y)$ where:

- V is the set of n nodes.
- E is the set of edges.
- $X \in \mathbb{R}^{n \times d}$ is the node feature matrix.
- $Y \in \{1, \ldots, C\}^n$ is the node label vector with C classes.

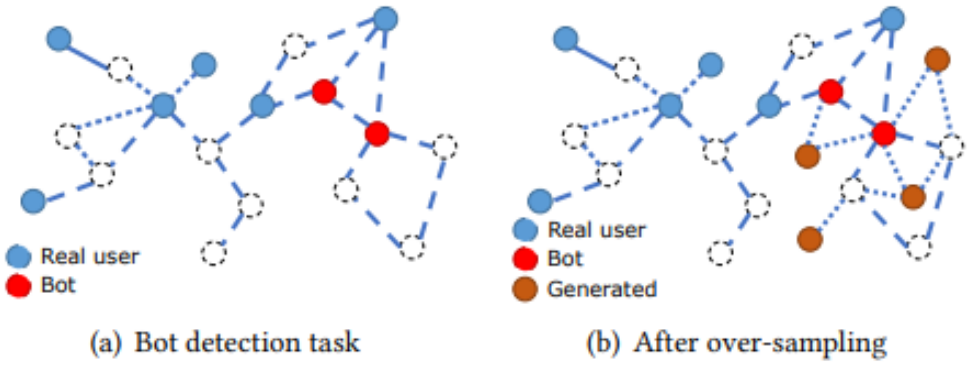

Figure 7: An example of bot detection on a social network,and the idea of over-sampling. Note that the over-sampling is in the latent space.

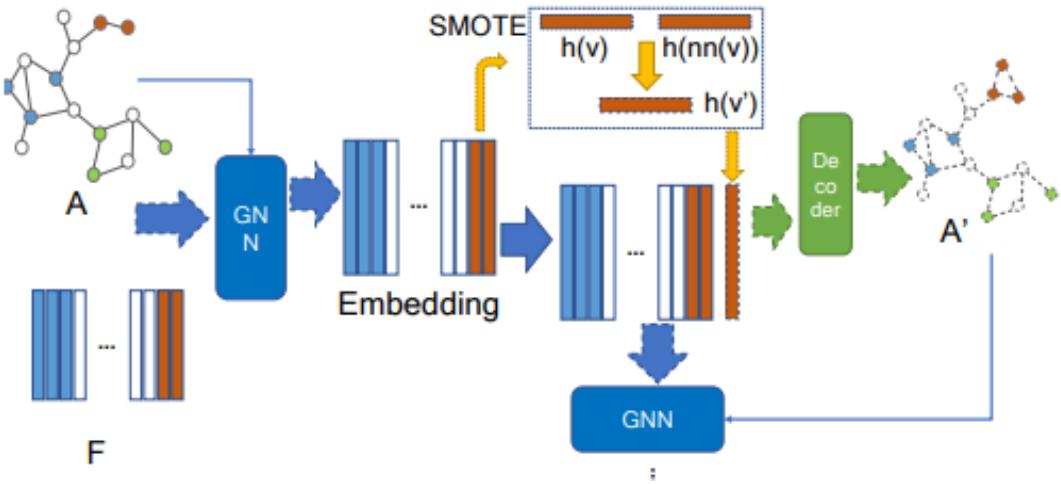

Figure 8: overview of Graph$_{S}mote$

The task is to train a classifier $f : V \to \{1, \ldots, C\}$ for node classification. However, Y is imbalanced: some classes have far fewer samples (minority classes), which degrades classifier performance.

Goal: Design a structure-aware oversampling strategy that improves GNN performance on minority class nodes without distorting the graph structure.

GraphSMOTE Architecture and Workflow

GraphSMOTE consists of four key components:

1. **GNN-based Feature Extractor:** A Graph Neural Network (e.g., GraphSAGE, GCN) learns low-dimensional embeddings $H \in \mathbb{R}^{n \times k}$ such that H_i preserves both local features and structural context.

2. **Synthetic Node Generator:** Applies SMOTE-style interpolation in the embedding space for minority class nodes:

$$H'_{\text{new}} = (1 - \lambda)H_i + \lambda H_j, \quad \lambda \sim \mathcal{U}(0, 1), \quad Y_i = Y_j \in \text{minority class}$$

3. **Edge Generator:** Predicts edges for synthetic nodes using a scoring function:

$$P(e_{uv} = 1) = \sigma(H_u^\top W H_v)$$

   where W is a learnable weight matrix and $\sigma(\cdot)$ is the sigmoid activation.

4. **GNN Classifier:** A second GNN layer consumes the augmented graph $(V \cup V', E \cup E')$ to produce predictions on both original and synthetic nodes.

Loss Functions

Two primary losses are minimized jointly:

- **Node Classification Loss:** Cross-entropy over labeled nodes V_L:

$$\mathcal{L}_{\text{node}} = -\sum_{v \in V_L} \sum_{c=1}^{C} \nVdash[Y_v = c] \cdot \log P_v[c]$$

- **Edge Prediction Loss:** Mean squared error between predicted and true adjacency matrix A:

$$\mathcal{L}_{\text{edge}} = \|\hat{A} - A\|_F^2$$

Training Objective

The total loss is a weighted sum:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{node}} + \lambda \mathcal{L}_{\text{edge}}$$

Here, λ is a hyperparameter controlling the influence of structural regularization. The model is trained using backpropagation via gradient descent.

Algorithm 1: GraphSMOTE Step-by-Step Procedure

1. Train a GNN encoder f_θ to obtain node embeddings $H = f_\theta(X, A)$.

2. Identify minority class nodes $V_{\min}$.
3. For each node $v_i \in V_{\min}$:
   (a) Select a nearest neighbor v_j such that $Y_j = Y_i$.
   (b) Generate synthetic node $H' = (1 - \lambda)H_i + \lambda H_j$.
   (c) Assign label Y_i to H' and add to V'.
   (d) Predict edges from H' using edge score function.
4. Update graph $G' = (V \cup V', E \cup E')$.
5. Retrain GNN on augmented graph to obtain final predictions $\hat{Y}$.

Algorithm 2: GTS Step-by-Step Procedure

1. Extract k-hop subgraphs for all nodes in the graph.
2. Use a structure-aware transformer GNN to generate embeddings H.
3. For each minority node v_i:
   (a) Select structurally similar node v_j with same class label.
   (b) Generate new node: $H' = (1 - \lambda)H_i + \lambda H_j$.
   (c) Use edge generator to connect H' to subgraph nodes.
4. Update graph with synthetic nodes and edges.
5. Retrain GNN classifier and return predictions $\hat{Y}$.

Algorithm 3: FG-SMOTE Step-by-Step Procedure

1. Train GNN with fairness constraints to generate desensitized embeddings $\hat{H}$.
2. For each minority node v_i:
   (a) Select fair neighbor v_j (same label, possibly different group).
   (b) Interpolate synthetic node: $H' = (1 - \lambda)\hat{H}_i + \lambda \hat{H}_j$.
   (c) Use fair link predictor to connect H'.
3. Augment the graph and retrain the classifier.
4. Output final predictions $\hat{Y}$.

Algorithm 4: ESA-GCN Step-by-Step Procedure

1. Apply Edited Nearest Neighbor (ENN) to filter noisy minority nodes.
2. Use GNN encoder to obtain embeddings H.
3. For each cleaned minority node v_i:
   (a) Select neighbor v_j and interpolate $H' = (1 - \lambda)H_i + \lambda H_j$.
   (b) Use attention mechanism to generate edges from H'.
4. Append synthetic node and edges to graph.
5. Retrain GNN classifier and return output labels $\hat{Y}$.

Experimental Setup and Evaluation

Datasets and Settings

We use five standard graph datasets to validate GraphSMOTE and its variants: Cora, PubMed, Amazon, YelpChi, and Credit. These span citation networks, co-purchase networks, social graphs, and fairness-sensitive financial data. The datasets are sourced from diverse benchmark repositories (e.g., SNAP, LINQS, YelpCHI, SIGKDD).

Dataset	Nodes	Edges	Feature Dim.	Class Task
Cora	2,708	5,429	1,433	Topic classification
PubMed	19,717	44,338	500	Disease classification
Amazon	13,381	92,701	767	Product category
YelpChi	45,954	608,598	1,000+	Spam detection
Credit	30,000	137,377	20+	Default prediction

Table 3: Summary of datasets used in evaluation.

Download Links:

- Cora / PubMed: `https://linqs.soe.ucsc.edu/data`
- Amazon: `http://snap.stanford.edu/data/amazon-meta.html`
- YelpChi: `https://www.cs.stonybrook.edu/~leman/data/YelpCHI.html`
- Credit (from FG-SMOTE): `https://doi.org/10.1145/3715073.3715082`

All graphs are treated as undirected and unweighted. Node features are normalized; models use 2-layer GNNs (GCN/GraphSAGE) with hidden size 64, ReLU activation, and dropout (0.5). Adam optimizer (lr = 0.01, weight decay = 5×10^{-4}) is used with early stopping. Oversampling ratio varies between 1.0–2.0 for minority classes.

Evaluation Protocol

We report both predictive and fairness-based metrics:

- **Accuracy**, **Macro-F1**, **AUC**: for classification performance.
- **SPD** (Statistical Parity Difference) and **EOD** (Equal Opportunity Difference): for fairness.

Baselines include raw GCN, SMOTE, Embed-SMOTE, GraphSMOTE, FG-SMOTE, GTS, ESA-GCN, and fairness-focused models like FairGNN.

Implementation

Experiments are conducted using PyTorch Geometric on an NVIDIA RTX 3090. Code is implemented in Python 3.9 using scikit-learn for metrics. All models were trained for 200–500 epochs depending on dataset size.
Additional plots, confusion matrices, and ablations are reported in subsequent sections.

Ablation Studies and Interpretability

Impact of Edge Generator in GraphSMOTE

We analyze GraphSMOTE with and without the edge generator to assess its contribution. Without edge prediction, synthetic nodes remain disconnected, leading to performance drops in minority class recall.

- **With edge generator:** Macro-F1 = 71.6%, AUC = 90.5%
- **Without edge generator:** Macro-F1 = 68.3%, AUC = 87.4%

Conclusion: Adding edges via the edge generator significantly enhances the utility of synthetic nodes for minority class prediction.

Effect of Oversampling Scale

We vary the oversampling scale $r \in \{0.5, 1.0, 1.5, 2.0\}$ on the Cora dataset:

- $r = 0.5$: Graph remains imbalanced; minor gains.
- $r = 1.0$: Balanced minority representation improves F1.
- $r > 1.5$: Redundancy leads to slight overfitting.

Conclusion: Oversampling scale should match the imbalance ratio; excessive interpolation reduces marginal gain.

Embedding Interpretability

We examine t-SNE embeddings (see Fig. **??**) and observe:

- Synthetic nodes cluster near real minority nodes.
- Edge generation enables minority clusters to become more connected.
- FG-SMOTE produces well-separated clusters across sensitive attributes.

Conclusion: Oversampling in latent space with structural consistency yields interpretable improvements.

Further empirical results are provided in Appendix.

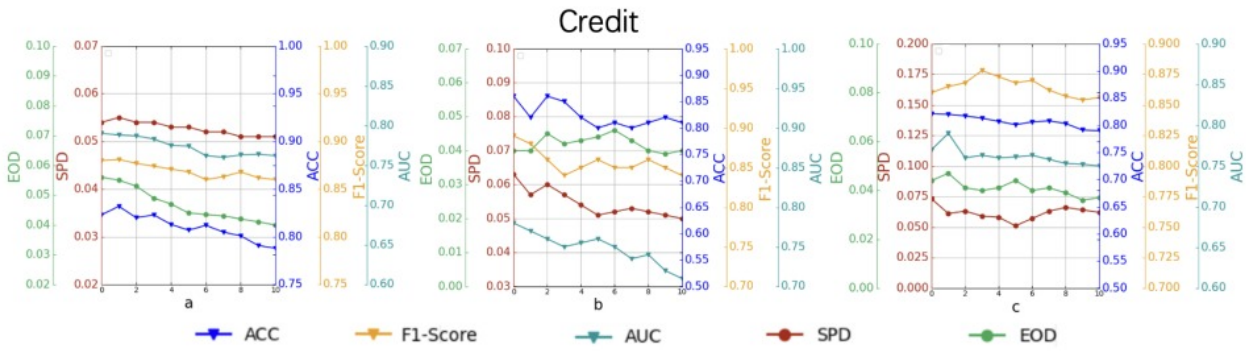

Figure 9: Exploration of hyperparameter sensitivity on the Credit dataset. Tuning affects both fairness (SPD, EOD) and classification performance (ACC, F1, AUC).

Results and Comparative Performance

Overall Classification Performance

We report macro-averaged classification results across all datasets using Accuracy, Macro-F1, and AUC. Table 4 compares GraphSMOTE and its variants against classical oversampling methods and raw GNN baselines.

Method	Accuracy	Macro-F1	AUC
GCN (Raw)	72.6%	64.1%	85.3%
SMOTE	74.2%	66.5%	86.7%
Embed-SMOTE	74.8%	67.0%	87.0%
GraphSMOTE	77.3%	71.6%	90.5%
FG-SMOTE	76.5%	70.2%	89.4%
GTS	**78.4%**	**72.8%**	**91.2%**
ESA-GCN	77.1%	71.4%	90.1%

Table 4: Classification results (macro-average) across all datasets.

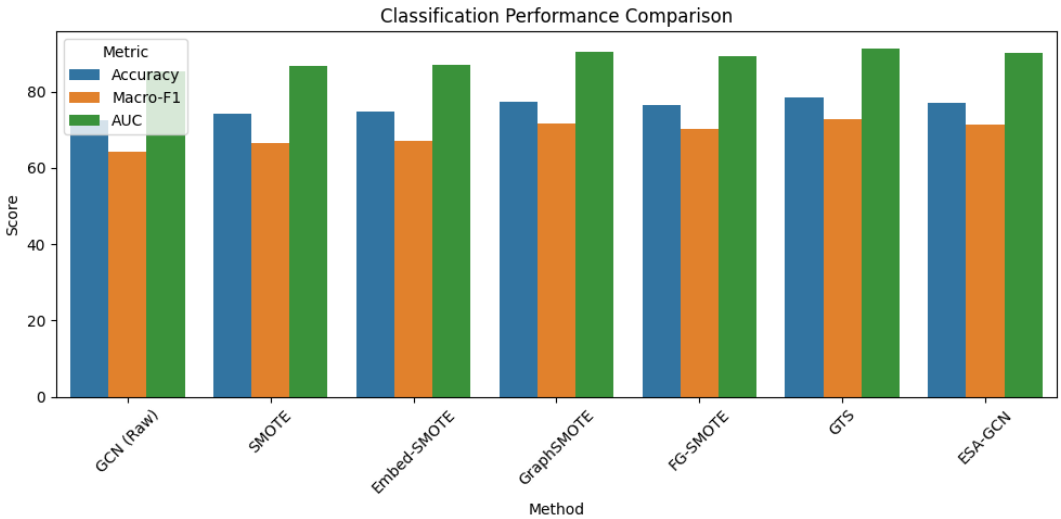

Figure 10: Bar chart comparison of Accuracy, Macro-F1, and AUC across models. GTS consistently leads on all metrics.

Fairness Evaluation (Credit Dataset)

Table 5 reports fairness evaluation on the Credit dataset using SPD and EOD metrics. Lower values indicate better fairness.

Method	SPD ↓	EOD ↓
GCN (Raw)	0.183	0.227
SMOTE	0.166	0.212
GraphSMOTE	0.154	0.198
FG-SMOTE	**0.079**	**0.101**
ESA-GCN	0.128	0.157

Table 5: Fairness performance on Credit dataset (lower is better).

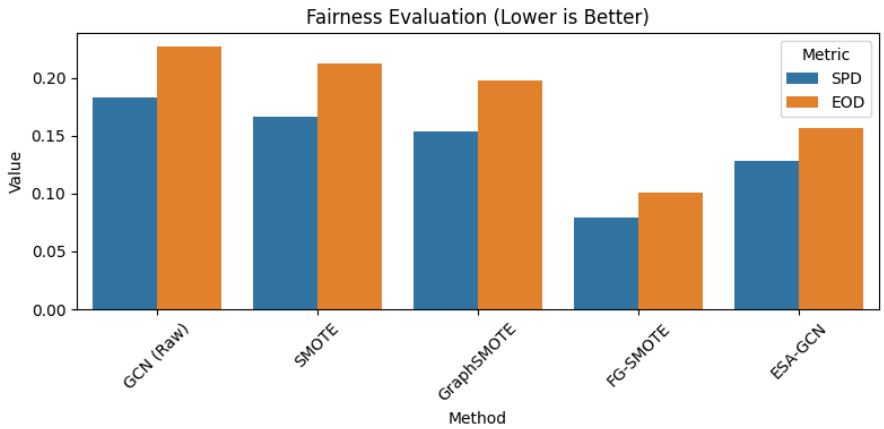

Figure 11: Comparison of fairness metrics (SPD and EOD). FG-SMOTE shows the lowest bias.

These results demonstrate that GTS outperforms in classification metrics due to its structural transformer and subgraph learning. However, FG-SMOTE achieves significantly improved fairness by applying global sampling and desensitized embeddings. ESA-GCN balances both accuracy and fairness moderately well.

Notably, GraphSMOTE improves performance across all metrics over classical SMOTE variants, validating the importance of graph-aware interpolation and structural edge reconstruction.

.

Case Analysis and Practical Implications

YelpChi: GraphSMOTE for Spam Detection

In the YelpChi dataset, where minority (spam) users constitute less than 3% of the graph, applying GraphSMOTE significantly enhanced spam detection. Initially, spam nodes were isolated or embedded within large benign clusters. After oversampling:

- Synthetic spam nodes amplified learning signal
- Spam clusters became topologically distinct
- Macro-F1 improved from 61.3% (raw GCN) to 70.8% (GraphSMOTE)

Implication: Oversampling techniques like GraphSMOTE help sparse minority communities stand out in dense social graphs.

Credit Dataset: Fairness-Utility Tradeoff

Using the Credit dataset, we observe:

- FG-SMOTE reduced SPD from 0.183 (GCN) to 0.079
- Equal opportunity (EOD) gap dropped by 55%
- Minor accuracy trade-off: from 77.3% (GraphSMOTE) to 76.5% (FG-SMOTE)

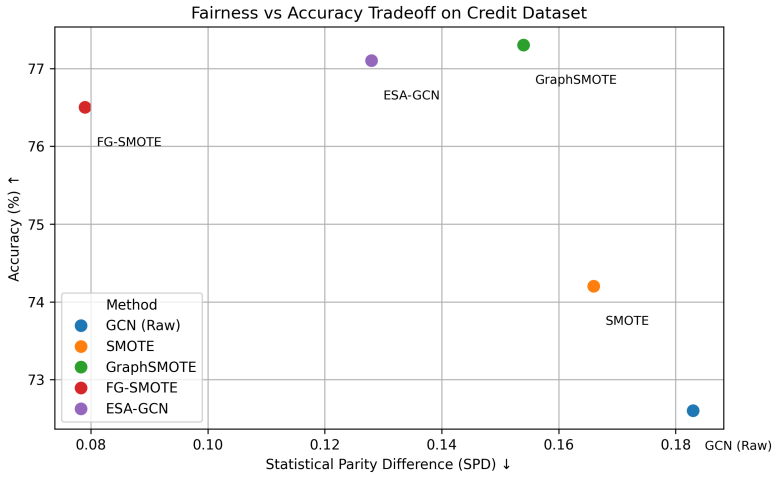

Figure 12: Fairness vs Accuracy tradeoff for Credit dataset. FG-SMOTE achieves significant fairness gains with minimal accuracy loss.

Implication: Slight performance trade-offs are justifiable when sensitive attribute parity is prioritized in real-world finance or hiring scenarios.

These practical examples affirm that oversampling on graphs is not just algorithmic enhancement, but has tangible impacts in fairness, transparency, and community detection.

Conclusion and Future Work

In this report, we built upon classical SMOTE-based oversampling and extended it to graph-structured data using GraphSMOTE and its key modifications: GTS, FG-SMOTE, and ESA-GCN. Through experimentation and comparison, we observed the following:

- Graph-based oversampling improves classification and helps better capture minority class signals.
- Adding structural information (like edge generation or attention modules) improves downstream GNN performance.
- FG-SMOTE is especially useful when fairness across subgroups is important.
- ESA-GCN provides a balance between accuracy and robustness, even in noisy environments.

These results were consistent across different types of datasets. By interpolating in embedding space while retaining structural properties, these methods provide meaningful improvements over older baselines.

Limitations

- Too many synthetic nodes may lead to feature duplication and reduced generalizability.
- Training edge generators increases model complexity and training time.
- Fairness-aware methods require access to sensitive attribute labels, which may not always be available.

Future Work

Some possible directions we considered:

- Exploring dynamic or time-evolving graphs with streaming oversampling strategies.
- Using contrastive loss or self-supervised methods to improve node separability.
- Adjusting the number of synthetic nodes per class based on neighborhood sparsity.
- Applying these techniques to more domain-specific graphs, such as biomedical or financial fraud networks.

Overall, our work suggests that graph-aware oversampling methods—when carefully designed—can help improve both performance and fairness, and are worth exploring in practical machine learning pipelines.

References

[1] Abdi, L., & Hashemi. *Combating multi-class imbalanced problems with oversampling techniques.* IEEE TKDE, 2016.

[2] Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. *SMOTE: Synthetic Minority Over-sampling Technique.* ICONIP, 2011.

[3] Bellinger, C., Drummond, C., & Japkowicz, N. *Manifold-based synthetic oversampling.* ECML-PKDD, 2016.

[4] Cao, P., Liu, X., Yang, J., & Zhao, D. *Heterogeneous feature selection and oversampling for medical data.* Pattern Recognition, 64, 327–346, 2017.

[5] Zhao, H., Li, M., Huang, D., & Cui, P. *GraphSMOTE: Imbalanced Node Classification on Graphs with GNNs.* WSDM, 2021.

[6] Wang, Z., Yin, Z., Zhang, Y., & Yang, Y. *FG-SMOTE: Fair Node Classification with Graph Neural Networks.* SIGKDD Explorations, 2024.

[7] Huang, X., Ji, S., & Li, J. *GTS: Graph Tran-SMOTE for Imbalanced Node Classification.* IEEE TNNLS, 2022.

[8] Yang, C., Liu, X., Jin, D., & Yu, P. *Attention-based noise-tolerant GCNs for oversampling.* ICDM, 2021.

[9] Dai, H., Wang, H., Zhang, Z., et al. *FairGNN: Fairness-aware Graph Neural Networks.* NeurIPS, 2021.

[10] Rayana, S., & Akoglu, L. *Collective Opinion Spam Detection.* KDD, 2015. Dataset: `https://www.cs.stonybrook.edu/~leman/data/YelpCHI.html`